

Continuation Must Preserve Revisability, Not Truth

The Failure of Terminal Facts in Machine-Recoverable Work State

Mitchell D. McPhetridge Independent Researcher · September 2026

Abstract

A continuity architecture is usually judged by what survives interruption. If a machine reader can recover the current task, relevant sources, prior decisions, unresolved contradictions, and next action after the previous runtime disappears, continuity appears to have succeeded.

This paper argues that this test is incomplete.

A system can recover its prior working state perfectly and still fail at continuation. The failure occurs when an earlier inference has entered durable state as fact. Once the classification survives, later runtimes no longer encounter the original uncertainty. They encounter a structured object whose downstream relationships may make it look increasingly established. The architecture has preserved not merely information but the consequences of having believed it.

This creates a failure mode distinct from ordinary hallucination. A hallucination is generated once. A continuation error survives replacement of the generator.

The central claim is:

Truth should not be a terminal state in a continuity architecture. Continuation must preserve revisability, not truth.

The architecture need not decide whether a proposition is true. It must preserve enough of the proposition's epistemic history that a later reader can determine what was claimed, what supported it, what was inferred rather than observed, what alternatives were available, what depended on it, and what would justify changing it.

The distinction is consequential for systems that preserve work state rather than documents alone. Persistent task state can convert an erroneous machine inference into routing, priority, human attention, and future evidence collection. A false proposition may therefore become more operationally influential without becoming more evidentially supported.

The paper defines terminal-fact failure, inherited-premise amplification, trajectory capture, and synthetic corroboration; proposes a revisable claim object; and gives experiments for determining whether continuity preserves a working history or merely makes yesterday's mistake easier to resume.

1. The Problem

Consider a machine-assisted project interrupted on Monday and resumed on Thursday by a fresh model with no conversational memory.

A useful continuity system might preserve:

- the active problem;
- the relevant documents;
- current claims;
- decisions already made;
- unresolved contradictions;
- work already attempted;
- priorities;
- constraints;
- and the next action.

The Thursday runtime can therefore resume where Monday's runtime stopped.

That is normally counted as success.

Now change one fact.

On Monday the model inferred that supplier B had discontinued a component. The source did not actually say this. It said that one configuration containing the component had been discontinued.

The distinction was missed.

The machine recorded:

Supplier B discontinued component X.

Several later decisions followed:

Component X unavailable → supplier B rejected → redesign required → evaluate suppliers C and D.

The human, seeing a coherent work map, spent Tuesday comparing C and D.

On Thursday a new runtime loads the persisted state. It does not make Monday's mistake.

It does not have to.

Monday made it for Thursday.

The new runtime sees supplier B as rejected, the redesign as active, comparisons of C and D as completed work, and the next action as choosing between them. Nothing about the recovered state necessarily requires it to reopen the original supplier

document.

The continuation system has worked mechanically.

It has failed epistemically.

2. Continuity Is Not Repetition

A useful distinction is:

continuity $\neq$ reproduction of prior belief

A system that restores yesterday's conclusion is not necessarily restoring yesterday's work.

The work also contained the conditions under which that conclusion was reached.

If those conditions disappear while the conclusion survives, the representation has compressed away the very information needed to revise itself.

The stronger requirement is:

continuity = recoverability + revisability

Recoverability asks:

Can another reader reconstruct the prior working position?

Revisability asks:

Can another reader determine why that position existed and what would permit it to change?

A system that satisfies the first but not the second is a persistence system, not yet a safe continuation system.

3. The Terminal-Fact Failure

Call a proposition terminal when downstream operation can rely on it without retaining a machine-recoverable path back to its epistemic basis.

This does not require a literal field named truth.

The same failure occurs with labels such as:

fact

confirmed

resolved

established

known

closed

or simply with an unqualified sentence inserted into the canonical work state.

The dangerous operation is not the word. It is the collapse:

$$(claim, basis, uncertainty) \rightarrow (fact)$$

The left side is revisable.

The right side is merely reusable.

That difference is small when a document is read once. It becomes large when the output determines the initial conditions of later machine reasoning.

4. Why Provenance Alone Is Insufficient

Provenance is necessary.

It is not enough.

Suppose the system records:

Component X was discontinued. Added by Runtime 14, 2026-09-18.

The provenance may be perfect. We know exactly which runtime wrote the statement and when.

We still do not know whether Runtime 14:

- quoted a source;
- paraphrased a source;
- inferred the proposition from several sources;
- accepted it from the human;
- inherited it from Runtime 13;
- generated it without retrieval;
- or converted a possibility into a declarative sentence.

The provenance of an artifact answers:

Where did this artifact come from?

The epistemic basis of a claim answers:

Why was this proposition admitted?

Those are different questions.

MPLPB already treats provenance and truth as separate questions: a librarian can return a false paper and a versioning system can preserve one perfectly; what

survives structurally is the history of claims and replacements, not a guarantee of correspondence with reality.

MPLPB_Cost_of_Visibility_v2(5).txt

The same distinction becomes more important when what is being preserved is not merely a paper but an operating state.

5. Work State Changes the Error

Ordinary knowledge persistence preserves propositions.

Work-state persistence can preserve the effects of propositions.

Let p be an erroneous proposition.

A simple corpus may contain:

$$p$$

A work-state system may contain:

$$p \rightarrow d_1 \rightarrow d_2 \rightarrow t_1 \rightarrow t_2$$

where:

- d_1 is a decision;
- d_2 is a rejected alternative;
- t_1 is the current task;
- t_2 is the next action.

The proposition has acquired an operational shadow.

Deleting or correcting p therefore does not necessarily correct:

$$d_1, d_2, t_1, t_2$$

This is the first major failure mode.

FM-C1: Orphaned Consequence

A premise is corrected but decisions, priorities, exclusions, or tasks derived from it remain active.

The corpus now disagrees with its own trajectory.

A correction mechanism that edits claims but does not invalidate downstream work is incomplete.

6. Inherited-Premise Amplification

A second failure is subtler.

Runtime A makes an unsupported inference p .

Runtime B receives p as state and produces q .

Runtime C receives p and q , and produces r .

The visible structure is now:

$$p \rightarrow q \rightarrow r$$

Three coherent objects exist.

There is still only one independent epistemic event.

The architecture must not confuse derivational multiplication with evidential multiplication.

This resembles epistemic laundering in machine-derived corpora: repeated machine generations can make an assertion structurally respectable without introducing an independent source. Origin depth was introduced in MPLPB specifically to expose that kind of chain; it measures derivational distance, not reliability.

Networked_MPLPB(3).pdf

For work state, the corresponding danger is stronger because the descendants may be decisions rather than merely documents.

Call this:

FM-C2: Inherited-Premise Amplification

Repeated continuation increases the structural footprint of a proposition without increasing its independent evidential support.

A useful invariant follows:

$$\textit{descendant count} \neq \textit{evidence count}$$

Ten decisions derived from one inference do not constitute ten reasons to believe the inference.

7. Synthetic Corroboration

Suppose three fresh runtimes independently inspect the same persisted work state.

All three conclude that supplier C is the best remaining choice.

This can look like agreement among three models.

But if all three began from the inherited premise that supplier B was unavailable, they did not independently establish the condition that shaped the answer.

They independently reasoned inside the same inherited error.

Call this:

FM-C3: Synthetic Corroboration

Several machine readers appear to agree because they share an unexamined inherited premise.

The number of readers is irrelevant if the epistemic root is shared.

Formally, if outputs $o_1, o_2, \dots, o_n$ all depend on inherited premise p ,

$$o_i \leftarrow p$$

then agreement among the o_i does not independently corroborate p .

This is especially dangerous in multi-agent systems because architectural multiplicity can be mistaken for epistemic independence.

It is not.

8. Trajectory Capture

A persistent work state does something a reference library does not.

It tells the next participant where to look next.

That is useful. It is also a control surface.

Suppose an erroneous inference creates:

Current problem: replace component X. Completed: supplier B eliminated. Priority: compare C/D. Next action: obtain C pricing.

The human returning to the project does not receive an explicit command to believe that B is impossible.

The interface merely makes B irrelevant.

This is enough.

Attention is finite. A system that determines what is active, resolved, blocked, or next can influence human investigation without possessing formal authority over the human.

Call this:

FM-C4: Trajectory Capture

An inherited machine judgment alters subsequent human attention such that the conditions capable of falsifying the judgment become less likely to be revisited.

The feedback loop is:

machine inference → persistent state → retrieval/routing → human attention → new work → stronger persistent state

No step requires deception.

No step requires autonomy.

No step requires the machine to be conscious, agentic, or malicious.

The failure arises from persistence plus finite attention.

9. The Human-Ratification Trap

Human review does not automatically solve this problem.

Suppose the machine says:

We established yesterday that supplier B discontinued X.

The human remembers discussing suppliers and clicks confirm.

The architecture now contains human ratification.

But the human may have confirmed the continuity record rather than independently verified the underlying proposition.

This distinction already matters for provenance systems. Origin-depth controls can expose machine-generation chains, but rubber-stamp ratification defeats the intended epistemic boundary if ratification is treated as fresh evidence rather than a recorded act. The network architecture explicitly recognizes that origin depth is not reliability.

Networked_MPLPB(3).pdf

Therefore:

human confirmation ≠ independent human verification

unless the system records what the human actually verified.

Call this:

FM-C5: Ratification Collapse

A human's acceptance of an inherited state is recorded as independent validation of the proposition contained in that state.

The system has converted interface compliance into epistemic evidence.

10. Revision Must Be First-Class

The solution is not to prohibit assertions.

People and machines have to make working judgments.

The mistake is treating a working judgment as an endpoint.

A claim object should preserve at least:

$$C=(P, B, O, D, S, A, V)$$

where:

- P = proposition;
- B = basis;
- O = origin;
- D = derivational depth or history;
- S = current operational status;
- A = live alternatives or objections;
- V = verification history.

This is deliberately not a truth score.

A claim may be sufficiently supported to guide work without the architecture claiming to know that it is true.

For example:

Proposition:

Supplier B discontinued component X.

Basis:

Inference from supplier bulletin 2026-41.

Source text:

Configuration AX-14 discontinued.

Origin:

machine inference

Independent verification:

none

Operational status:

working assumption

Alternatives:

component remains available outside AX-14

Dependents:

supplier-B rejection
redesign task
C/D comparison

Revision condition:
verify component-level availability with supplier catalog

The record does not prevent error.

It makes the error reachable.

That is the relevant property.

11. Supersession Is Better Than Correction

A continuation architecture should be reluctant to “fix” history.

If proposition p_1 was used on Monday and replaced by p_2 on Thursday, Thursday should not rewrite Monday to make it appear that p_2 was always known.

The useful relation is:

$$p_2 \text{ supersedes } p_1$$

while retaining:

$$p_1 \rightarrow \{d_1, d_2, t_1\}$$

This makes it possible to ask:

What work depended on the proposition we just replaced?

Smart Local MPLPB already applies this principle to taught knowledge: revision creates a replacement, retires rather than deletes the old page, records the supersession, and preserves both versions so the corpus can say when it changed its mind.

Smart_Local_MPLPB(20260922-121541).txt

A work-state system requires the same discipline one level deeper.

It must be able to say not only:

We changed our mind.

but:

These decisions were made while we believed the thing we changed our mind about.

12. Dependency-Aware Revision

Let the working state be a directed graph:

$$G=(V, E)$$

where vertices are claims, decisions, tasks, exclusions, priorities, and outputs.

An edge:

$$x \rightarrow y$$

means that y materially depended on x .

When x is superseded, the system should not automatically delete y .

It should mark the dependency as requiring reconsideration.

Define:

$$R(x) = \{y \mid x \rightsquigarrow y\}$$

where $R(x)$ is the revision surface: all reachable descendants whose validity or relevance may depend on x .

The size of $R(x)$ matters operationally.

A corrected typo may have:

$$|R(x)| = 0$$

A corrected foundational assumption may have:

$$|R(x)| = 417$$

Both are “one corrected object” in a naïve version log.

They are not equivalent continuation events.

13. Failure Modes

The architecture proposed here should be considered defective if it permits the following without making them machine-visible:

ID	Failure	Description
FM-C1	Orphaned consequence	A premise changes while dependent work remains silently operative.
FM-C2	Inherited-premise amplification	Repeated derivation increases apparent support without new evidence.
FM-C3	Synthetic corroboration	Multiple runtimes sharing one premise are mistaken for independent confirmation.
FM-C4	Trajectory capture	Persisted machine judgments redirect human attention away from their own falsifiers.
FM-C5	Ratification collapse	Human acceptance of state is treated as independent verification of its contents.
FM-C6	Terminal fact	A proposition becomes operationally unrevisable because its basis no longer travels with it.

ID	Failure	Description
FM-C7	Correction without replay	A premise is corrected but affected decisions are never reconsidered.
FM-C8	Status laundering	“working assumption” becomes “established” through repetition rather than evidence.
FM-C9	Confidence inheritance	A new runtime inherits prior confidence without independently recovering its basis.
FM-C10	History sterilization	Revision overwrites the old state and erases the conditions under which prior actions were reasonable.

These failures have a common structure.

The architecture preserved state better than it preserved the ability to interrogate state.

14. The Wrong Metric

A common continuation experiment asks a fresh runtime to resume a task and measures whether it reconstructs the prior working position.

That test rewards fidelity.

Fidelity is necessary.

It is not sufficient.

Imagine two systems.

System A reproduces 98% of the previous working state and repeats an embedded false assumption 98% of the time.

System B reproduces 90% of the working state but identifies the unsupported premise and reopens the affected branch 70% of the time.

Which system has better continuity?

If continuity means copying state, A wins.

If continuity means allowing work to survive replacement without freezing its mistakes, the answer is no longer obvious.

A continuation benchmark therefore needs at least two axes:

$$C = f(F, R)$$

where:

- F = reconstruction fidelity;
- R = retained revisability.

Optimizing F alone may make R worse.

That possibility should be tested rather than assumed away.

15. The Poisoned-Continuation Experiment

A simple experiment can test the claim.

Prepare a bounded project containing ten source documents and a multi-step decision task.

During the first session, introduce one plausible but false machine inference. Allow the system to work from it long enough that several decisions and next actions depend on it.

Then replace the runtime.

Compare three conditions:

A. Source continuity only. The new runtime receives the corpus and provenance but no prior work state.

B. Terminal work-state continuity. The new runtime receives claims, decisions, completed tasks, priorities, and next actions, but the false premise is represented as established state.

C. Revisable work-state continuity. The new runtime receives the same state, but propositions retain basis, derivation, alternatives, verification status, and dependency links.

Measure:

T_d = time to detect poisoned premise

P_c = probability of correction

D_e = number of downstream decisions examined after correction

H_a = human anchoring rate

E_i = independent evidence acquired before reaffirmation

The important comparison is B versus C.

If explicit work-state continuity improves resumption but makes the poisoned premise harder to discover, then continuity has introduced an epistemic cost.

If C retains the resumption advantage while restoring challenge and correction, revisability has earned its complexity.

If A performs as well as C, the richer architecture is unnecessary.

All three outcomes are useful.

16. What This Does Not Solve

Revisability does not establish truth.

A beautifully versioned falsehood remains false.

A proposition with impeccable provenance may still be wrong.

A human can independently verify something incorrectly.

A machine can correctly challenge a true proposition.

A dependency graph can omit a dependency.

A revision trigger can fire too often and make useful work impossible to stabilize.

The proposal is therefore deliberately narrower.

It does not solve epistemology.

It attempts to prevent a continuity architecture from pretending that epistemology has already been solved.

That boundary matters.

17. Continuation Without Belief

The strongest form of the proposal can be stated simply:

A successor runtime should inherit the previous runtime's work, not its obligation to believe the previous runtime.

This implies an asymmetry.

The architecture should make prior work cheap to recover.

It should not make prior conclusions expensive to question.

A useful continuation package therefore carries enough structure to answer two questions at once:

Where were we?

and

Why were we there?

If it answers only the first, it preserves location without preserving navigation.

18. Falsifiers

The argument in this paper is wrong, or overstated, if controlled experiments show that:

1. terminal fact labels do not measurably increase inherited-error persistence;
2. carrying epistemic basis does not improve error discovery over ordinary provenance;
3. dependency-aware revision does not improve reconsideration of affected work;
4. human operators are no more anchored by inherited work-state classifications than by source-only continuity;
5. multi-runtime agreement is not materially affected by shared inherited premises;
6. a materially simpler version-and-provenance system produces the same revisability;
7. the additional metadata creates enough cognitive or operational burden to make continuation worse overall.

The sixth result would be particularly important.

If ordinary provenance plus supersession is enough, a richer continuity layer should not be built merely because it can be described.

19. Conclusion

Persistence is easy to mistake for continuity because both answer the same superficial question:

Did it survive?

The harder question is:

What survived?

If a document survives, we have preservation.

If a working state survives, we have recoverability.

If a working state survives together with the basis, uncertainty, alternatives, dependency history, and mechanisms necessary to overturn it, we begin to have continuation.

A system that carries yesterday's mistake perfectly into tomorrow has demonstrated persistence, not epistemic continuity.

A system that accumulates descendants of that mistake has not accumulated evidence.

A system that presents those descendants to a human as the natural next steps can guide the human without ever possessing formal authority.

And a system that later corrects the premise but cannot identify what the premise changed has not actually corrected the work.

The design principle is therefore deliberately negative:

Truth should never be a terminal state in a continuity architecture.

Not because truth does not exist.

Not because all claims are equally uncertain.

Not because machines can never establish anything useful.

But because continuation is the wrong layer to declare the inquiry finished.

Its job is different.

It should preserve what was claimed, why it was admitted, what depended on it, what challenged it, what replaced it, and enough of that history that a future reader can disagree intelligently.

Continuation must preserve revisability, not truth.